Reflection on Group Assignment 2IRR00

Ignacy Świderski
Eindhoven University of Technology, The Netherlands
2272563, Group 24

1 Project Description and Scope

Our project is an accessible audiobook player designed for elderly users, addressing the common issue of overly complex interfaces and poorly defined user needs. Built in Java 21 with JavaFX 21 for the desktop Graphical User Interface (GUI), the system allows users to browse a local audiobook library, play and pause episodes, save listening progress, and adjust volume, playback speed, and a sleep timer. The system is fully implemented, including both the data model and the complete GUI, with unit tests covering all production code.

2 Project Reflection

On the technical side, I felt the project moved forward well, with testing and design decisions receiving consistent attention throughout development. The bigger challenge turned out to be the team dynamics. This subsections reflect on three key areas: process and collaboration, testing and quality assurance, and design principles.

2.1 Process and Collaboration

We relied on GitHub for version control, using branches and pull requests to coordinate contributions. However, I observed considerable variation in the quality and wholeness of contributions across the team. Some commits introduced untested code, lacked meaningful commit messages, or did not follow the agreed coding conventions. This required additional review effort and, in some cases, rework to bring that up to a standard that could be safely merged. The impact was felt most during integration phases, where inconsistent code made it harder to reason about the system as a whole and slowed down progress.
Reflecting on this, I believe that establishing explicit contribution standards early on, like for instance defining a *done*, requiring code review before merging, and agreeing on a shared coding style, is something I would prioritize in future projects, even within a group of friends. Having such constraints in place from the start would reduce the fuss of informal negotiation later.

2.2 Testing and Quality Assurance

Throughout development, I have written tests in parallel with the implementation, as reflected in the commit history. While the approach is not strictly test-driven, I have maintained the discipline of leaving no production code without tests. I was brainstorming test cases together with one other team member, which proved particularly valuable, as it covered edge cases I would likely have overlooked. This has reinforced my belief that testing is most meaningful when approached as a design activity rather than a verification step. Writing tests forces explicit reasoning about contracts, boundaries, and failure modes before the implementation is finalized. This aligns with what we've been told during the course.

2.3 Design Principles and Patterns

I have aimed to apply SOLID principles consistently across my contributions. In particular, I applied the *Single Responsibility Principle* by keeping each class focused on a single concern: `ProgressStore` handles only persistence, while `Library` manages the collection of audiobooks without knowing how they are stored. I also applied the *Dependency Inversion Principle* by depending on abstractions where appropriate. This made individual units easier to test.
One particular design choice was adopting a best-effort strategy for progress saving. Rather than guaranteeing that every state change is immediately written to disk, which could introduce latency on the UI thread, the application saves progress opportunistically, accepting the possibility of minor data loss in exchange for responsiveness. This felt like a reasonable compromise given our target user group.

3 Self-Reflection

My primary contribution to the project has been the entire data model layer – the `Episode`, `Audiobook`, `Library`, and `ProgressStore` classes, along with their unit tests. I also took on an informal leadership role within the group.
In terms of what I have learned, the experience with the team has showed how significantly team dynamics affect the quality of a project. Rather than placing the responsibility on others, I believe that I could have done more to communicate the expectations early. I learned that if you want a team to actually get things done, you have to stay on top of them. From now on, I realize that picking good people to work with is just as important as the technical work itself.
On a personal level, I recognize a tendency to postpone certain tasks rather than address them immediately. Going forward, I intend to work more consistently across the project timeline by breaking work into smaller milestones and scheduling short weekly planning sessions to track progress. I also intend to continue deepening my knowledge of Java, particularly in areas in which my experience remains limited so far.

4 Conclusion and Learnings

Reflecting on the completed project, my key takeaway is that the technical and human sides of software development are equally important. And that neglecting either has real consequences. Implementing the data model layer gave me an appreciation for thoughtful design choices, while the team experience showed the importance of clear expectations and collaboration. I intend to carry these lessons forward into future projects.